

Efficient 3D Object Classification with Orientation-Boosted Voxel Networks

Giuseppe D’Auria (2163508)
Department of Information Engineering
Università di Padova
giuseppe.dauria@studenti.unipd.it

Abstract—This work presents a PyTorch implementation of ORION (ORIENTATION-boosted vOxel Net), a 3D convolutional neural network for object classification that leverages orientation estimation as an auxiliary task. I replicate the original architecture from the publicly available codebase and propose ORION-Fast, an efficient variant achieving 48% parameter reduction while maintaining competitive accuracy. Experiments on ModelNet10 and ModelNet40 benchmarks demonstrate that the Fast variant achieves 93.50% and 86.63% classification accuracy respectively (using test-time voting), with accuracy gaps of only 0.22 and 0.93 percentage points compared to the baseline. Additionally, I evaluate the extended 4-layer architecture on ModelNet40, achieving 88.33% accuracy. A cross-architecture comparison with PointNet reveals that ORION-Fast achieves comparable accuracy with $7.2\times$ fewer parameters, confirming the parameter efficiency of voxel-based approaches with orientation boosting. The parameter reduction is achieved through systematic FC layer compression, validated by ablation studies.

Index Terms—3D Object Classification, Convolutional Neural Networks, Voxel Representation, Multi-task Learning, Model Compression.

I. INTRODUCTION

Three-dimensional object recognition has become increasingly important with the proliferation of depth sensors, LiDAR systems, and CAD modeling applications. Autonomous vehicles require robust 3D perception for obstacle detection, robotic manipulation systems need accurate object pose estimation, and augmented reality applications demand real-time 3D scene understanding. Unlike 2D image classification, 3D recognition must handle the additional complexity of object orientation, which significantly affects the feature representations learned by neural networks.

A fundamental challenge in 3D object classification is achieving rotation invariance. When an object is rotated, its voxel representation changes substantially, yet the semantic class should remain unchanged. Traditional approaches address this through extensive data augmentation, feeding multiple rotations of each object during training. However, Sedaghat et al. [2] demonstrated a more principled approach: treating orientation estimation as an auxiliary task during training improves classification accuracy. The key insight is that forcing the network to predict object orientation helps it learn more discriminative and rotation-aware features, even though orientation is not required at test time. This multi-task learning approach achieved state-of-the-art results on multiple

3D datasets with a relatively shallow architecture of only two convolutional layers.

This work presents four contributions:

- A complete PyTorch reimplement of ORION, based on the original codebase [1] and modernized with contemporary training practices.
- ORION-Fast, an efficient variant that reduces parameters by 48% through systematic fully-connected layer compression, informed by ablation studies on layer importance.
- A comprehensive experimental analysis on ModelNet10 and ModelNet40 benchmarks, including evaluation of the extended 4-layer architecture and detailed orientation prediction analysis.
- A cross-architecture comparison with PointNet [6], isolating the effect of data representation while controlling for training data and evaluation protocol.

My experiments show that significant parameter reduction is achievable with minimal accuracy degradation (less than 1 percentage point with voting), making the model more suitable for resource-constrained deployment scenarios such as embedded systems or edge devices.

The remainder of this paper is organized as follows. Section II reviews related work in 3D object classification and model compression. Section III describes the data preprocessing pipeline and network architectures in detail. Section IV presents experimental results including training dynamics, accuracy comparisons, and ablation studies. Section V presents a cross-architecture comparison with PointNet. Section VI provides concluding remarks and lessons learned.

II. RELATED WORK

Voxel-based Methods. The voxelization of 3D shapes into regular grids enables the direct application of 3D convolutions, analogous to 2D convolutions on images. Wu et al. [3] introduced 3D ShapeNets, representing objects as probability distributions over binary voxel grids using Deep Belief Networks. Their work also introduced the ModelNet benchmark, which has become the standard evaluation dataset for 3D classification. Maturana and Scherer [4] proposed VoxNet, a more compact 3D CNN operating on occupancy grids. VoxNet achieved good performance with only two convolutional layers and demonstrated real-time inference capability. These methods established voxel grids as an effective representation for

3D recognition, though they are limited by the cubic memory growth of voxel representations.

Multi-task Learning for 3D Recognition. ORION [2] extended VoxNet by adding orientation prediction as an auxiliary task. The key architectural innovation is the use of separate output heads for class and orientation prediction, with a combined loss function that equally weights both tasks. Critically, the orientation classes are not shared across object categories, each class has its own set of orientation bins. This prevents the network from learning absolute orientation features and instead encourages learning relative, class-specific orientation representations. The original implementation achieved 93.8% on ModelNet10 and demonstrated improvements across diverse datasets including LiDAR point clouds (Sydney Urban Objects), RGB-D data (NYUv2), and CAD models (ModelNet).

Alternative 3D Representations. Su et al. [5] proposed Multi-View CNN (MVCNN), rendering 3D objects from multiple viewpoints and aggregating 2D CNN features. Qi et al. [6] introduced PointNet, which processes raw point clouds directly using shared MLPs and a symmetric aggregation function (global max pooling) to achieve permutation invariance over unordered point sets. While these alternative representations achieve competitive or superior accuracy, voxel-based approaches remain relevant due to their conceptual simplicity and compatibility with standard convolution operations.

Model Compression. Han et al. [7] demonstrated that networks can be pruned to remove redundant connections with minimal accuracy loss. For 3D networks, parameter efficiency is particularly important due to the cubic growth of voxel representations. This work explores compression through architectural modifications.

III. METHOD

A. Data Preprocessing

The ModelNet dataset [3] provides 3D CAD models in OFF (Object File Format) mesh representation. Converting these meshes to voxel grids suitable for 3D convolutions requires a multi-step pipeline.

Mesh to Point Cloud. Surface points are sampled uniformly from mesh faces using area-weighted sampling. For each triangular face with vertices v_1, v_2, v_3 , we generate points using barycentric coordinates:

$$\mathbf{p} = (1 - \sqrt{r_1})\mathbf{v}_1 + \sqrt{r_1}(1 - r_2)\mathbf{v}_2 + \sqrt{r_1}r_2\mathbf{v}_3 \quad (1)$$

where $r_1, r_2 \sim U(0, 1)$. The number of points per face is proportional to face area, generating approximately 10,000 points per object to ensure adequate surface coverage.

Point Cloud to Voxel Grid. Points are first centered at the origin by subtracting the centroid. The point cloud is then scaled to fit within a $[-1, 1]^3$ cube while preserving aspect ratio:

$$\mathbf{p}' = \frac{\mathbf{p} - \mathbf{c}}{\max_i |\mathbf{p}_i - \mathbf{c}_i|} \quad (2)$$

where $\mathbf{c}$ is the centroid. The normalized point cloud is discretized into a $32 \times 32 \times 32$ binary occupancy grid, where each

voxel is set to 1 if at least one point falls within its bounds, and 0 otherwise. This resolution balances detail preservation with computational tractability.

Data Augmentation. Training samples are augmented with 12 discrete rotations around the vertical (z) axis at 30° increments, consistent with the original ORION implementation. This augmentation serves dual purposes: (1) increasing effective training data by $12\times$, and (2) providing ground-truth orientation labels for the auxiliary task. The rotation index $r \in \{0, 1, \dots, 11\}$ directly corresponds to the orientation class label.

Dataset Statistics. ModelNet10 contains 3,991 training and 908 test samples across 10 categories. ModelNet40 contains 9,843 training and 2,468 test samples across 40 categories. For ModelNet40, I use the automatically aligned version (ModelNet40-auto) provided with the original ORION codebase, which applies unsupervised alignment to obtain orientation labels. I reserve 10% of training data for validation using stratified sampling.

B. ORION Architecture

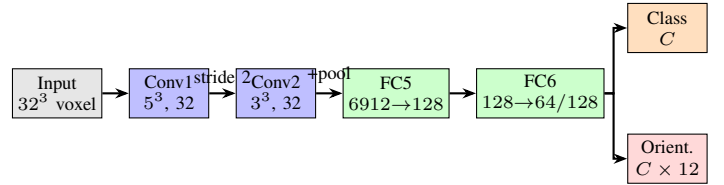


Fig. 1. ORION architecture overview. The network processes 32^3 voxel grids through convolutional layers (Conv1, Conv2) and fully-connected layers (FC5, FC6), producing class and orientation predictions via separate output heads. LeakyReLU activations and dropout are applied after each layer (not shown).

The baseline ORION architecture, illustrated in Fig. 1, extends VoxNet with a dual-head output structure for joint classification and orientation estimation.

Convolutional Feature Extraction. The input is a $32 \times 32 \times 32 \times 1$ binary occupancy grid. Conv1 applies 32 filters of size $5 \times 5 \times 5$ with stride 2 and no padding, producing $14 \times 14 \times 14 \times 32$ feature maps. LeakyReLU activation (negative slope 0.1) is applied. Conv2 applies 32 filters of size $3 \times 3 \times 3$ with stride 1 and no padding, followed by LeakyReLU and $2 \times 2 \times 2$ max pooling with stride 2. This yields $6 \times 6 \times 6 \times 32 = 6912$ features.

Fully-Connected Layers. The flattened 6912-dimensional feature vector passes through FC5, which reduces dimensionality to 128, followed by LeakyReLU and dropout (rate 0.3). FC6 further processes the representation, maintaining 128 dimensions in the baseline architecture (modified to 64 in the Fast variant). LeakyReLU and dropout are applied after FC6 as well.

Dual Output Heads. The classification head is a linear layer producing C logits, where $C = 10$ for ModelNet10 and $C = 40$ for ModelNet40. The orientation head produces $C \times O$ logits, where $O = 12$ is the number of orientation bins. Critically, orientations are not shared across classes, each class

has its own set of 12 orientation outputs. This design prevents the network from learning class-agnostic absolute orientation features and instead encourages class-specific relative orientation representations.

Loss Function. The combined loss balances classification and orientation tasks:

$$\mathcal{L} = (1 - \gamma)\mathcal{L}_{\text{cls}} + \gamma\mathcal{L}_{\text{ori}} \quad (3)$$

where $\mathcal{L}_{\text{cls}}$ and $\mathcal{L}_{\text{ori}}$ are cross-entropy losses for classification and orientation respectively. Following the original paper, I set $\gamma = 0.5$.

Parameter Count Discrepancy. The original paper reports 910K parameters for the 2-layer architecture and 4M for the 4-layer extended version. However, the actual codebase yields 931,539 and 3,290,341 parameters respectively. This discrepancy likely arises from differences in reported versus implemented layer dimensions. For this work, I adhere strictly to the implementation in the public repository.

C. ORION-Fast Architecture

My efficiency-focused variant, ORION-Fast, targets the fully-connected layers which contain the majority of parameters. The design is motivated by empirical observations and literature findings that FC layers in classification networks are often over-parameterized [7]. In the baseline architecture, the parameter distribution is highly skewed: FC5 alone contains 884,864 parameters (95% of total), while the convolutional layers contain only 31,712 parameters (3.4%). This suggests that compression efforts should focus on fully-connected layers for maximum impact.

Architecture Modifications. ORION-Fast reduces FC6 from 128 to 64 neurons, yielding the layer configuration shown in Tab. I. This single modification reduces total parameters by approximately 48%, from 931K to 482K for ModelNet10. The convolutional layers remain unchanged to preserve feature extraction capacity. Additionally, I apply label smoothing ($\epsilon = 0.01$) to the classification loss for the Fast variant to improve generalization:

$$\mathcal{L}_{\text{cls}} = - \sum_{c=1}^C \tilde{y}_c \log(p_c), \quad \tilde{y}_c = (1 - \epsilon)y_c + \frac{\epsilon}{C} \quad (4)$$

where y_c is the one-hot label and p_c is the predicted probability.

TABLE I
LAYER-WISE PARAMETER COMPARISON BETWEEN ORION BASIC AND ORION-FAST ARCHITECTURES. VALUES FOR CLASSIFICATION AND ORIENTATION HEADS VARY BY DATASET (MN10/MN40).

Layer	Basic	Fast
Conv1 (5^3 , 32 filters)	4,032	4,032
Conv2 (3^3 , 32 filters)	27,680	27,680
FC5 (6912 $\rightarrow$ 128)	884,864	884,864
FC6 (128 $\rightarrow$ 128/64)	16,512	8,256
Class Head	1,290/5,160	650/2,600
Orientation Head	15,480/61,920	7,740/30,960
Total (MN10/MN40)	931K/946K	482K/489K
Reduction	–	48%

Design Rationale and Failed Attempts. I explored several compression strategies before arriving at the Fast configuration:

- *SE Attention [8]:* Adding SE blocks after convolutional layers increased parameters by 8% without improving accuracy.
- *FC5 Compression:* Reducing FC5 from 128 to 64 neurons caused accuracy drops exceeding 3%, indicating this layer performs critical dimensionality reduction from 6912-D features.
- *Deeper Networks:* A third convolutional layer increased parameters without accuracy improvement, likely due to overfitting.
- *FC6 Compression:* Reducing FC6 to 64 neurons showed minimal accuracy impact ($<1\%$ on MN10); further reduction to 32 degraded accuracy by 2.1%.
- *Dropout Tuning:* Increasing dropout beyond 0.3 degraded performance; I retain the original rate.

These experiments demonstrate that FC5 is critical while FC6 exhibits redundancy, guiding the final Fast configuration.

D. Extended Architecture

I also implement the extended 4-layer ORION architecture from the original codebase. Importantly, this architecture differs substantially from the 2-layer Basic version:

- Conv1: 32 filters, $3 \times 3 \times 3$ kernel (not $5 \times 5 \times 5$), stride 2
- Conv2: 64 filters (not 32), $3 \times 3 \times 3$ kernel, stride 1
- Conv3: 128 filters, $3 \times 3 \times 3$ kernel, stride 1
- Conv4: 256 filters, $3 \times 3 \times 3$ kernel, stride 1
- MaxPool: Applied after Conv4 (not Conv2)
- Dropout: Progressive rates 0.2, 0.3, 0.4, 0.6 (higher final dropout)

This architecture contains 3,290,341 parameters and is evaluated on ModelNet40 to provide a complete comparison with published results.

E. Test-Time Voting

Following the original ORION evaluation protocol, I implement test-time voting for fair comparison with published

results. During testing, each object is evaluated at all 12 rotation angles, and the class scores are aggregated:

$$c^* = \arg \max_k \sum_{r=0}^{11} S_k(x_r) \quad (5)$$

where $S_k(x_r)$ is the softmax score for class k given input x at rotation r . This voting procedure improves accuracy by 0.5–1% compared to single-shot evaluation on a random rotation. All reported test accuracies use voting unless otherwise noted.

F. Training Configuration

All models are trained for 50 epochs using stochastic gradient descent (SGD) with momentum 0.9 and weight decay 10^{-4} . The initial learning rate is 0.01, reduced by factor 10 at epochs 20 and 40. Batch size is 32. Training uses a single NVIDIA A40 GPU (DEI CLUSTER) with PyTorch’s default weight initialization.

IV. RESULTS

A. Classification Accuracy

TABLE II
CLASSIFICATION ACCURACY (%) ON MODELNET BENCHMARKS WITH TEST-TIME VOTING. PARAMETERS IN THOUSANDS. GAP INDICATES ACCURACY DIFFERENCE FROM BASIC.

Model	Dataset	Params	Val	Test	Gap
Basic	MN10	931K	93.98	93.72	–
Fast	MN10	482K	93.50	93.50	-0.22
Basic	MN40	946K	87.48	87.56	–
Fast	MN40	489K	86.91	86.63	-0.93
Extended	MN40	3.29M	88.10	88.33	+0.77

Tab. II presents classification accuracy for all configurations using test-time voting. The baseline ORION implementation achieves 93.72% on ModelNet10 and 87.56% on ModelNet40 (using auto-aligned orientations). The extended 4-layer architecture achieves 88.33% on ModelNet40 with 78.24% orientation accuracy.

ORION-Fast achieves 93.50% on ModelNet10 and 86.63% on ModelNet40, representing accuracy gaps of only 0.22 and 0.93 percentage points respectively. The extended model achieves 88.33%, demonstrating that additional convolutional capacity does improve accuracy on the more challenging 40-class task.

B. Effect of Test-Time Voting

TABLE III
IMPACT OF TEST-TIME VOTING ON CLASSIFICATION ACCURACY (%).

Model	Dataset	Single-Shot	Voting	Gain
Basic	MN10	93.17	93.72	+0.55
Fast	MN10	92.73	93.50	+0.77
Basic	MN40	86.79	87.56	+0.77
Fast	MN40	85.74	86.63	+0.89
Extended	MN40	87.48	88.33	+0.85

Test-time voting aggregates predictions across all 12 rotations per test object, as described in Equation 5. Table III compares single-shot evaluation (random rotation) with voting. Voting consistently improves accuracy by 0.5–0.9%, with slightly larger gains for the Fast variant. This suggests the smaller model benefits more from aggregating multiple views, partially compensating for reduced single-shot capacity.

C. Training Dynamics

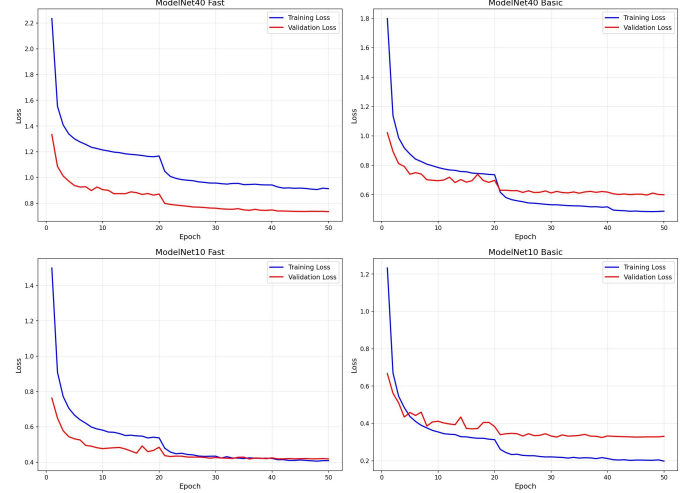


Fig. 2. Training (blue) and validation (red) loss curves over 50 epochs. Learning rate reductions at epochs 20 and 40 produce visible loss drops.

Fig. 2 shows training and validation loss curves for all configurations. Several observations emerge:

Convergence Behavior. All models exhibit rapid initial convergence during epochs 1–10, where the loss decreases from initial values around 1.5–2.2 to approximately 0.4–0.8. The learning rate schedule produces clear loss reductions at epochs 20 and 40, enabling the optimizer to settle into sharper minima.

Generalization Gap. The gap between training and validation loss is larger for Fast variants, particularly on ModelNet40. For MN40 Basic, the final training loss is approximately 0.49 versus validation loss of 0.60. This suggests the reduced capacity of Fast models provides implicit regularization but may limit representational power for the harder 40-class task.

ModelNet40 Complexity. MN40 models show higher absolute loss values due to the $4\times$ increase in classes (40 vs 10) and the corresponding expansion of the orientation prediction space from 120 to 480 orientation classes. The curves also exhibit more variance, reflecting the increased difficulty.

D. Orientation Prediction Analysis

The orientation prediction task is central to ORION’s multi-task learning approach. Although orientation is not used at test time, its prediction during training guides the network toward learning rotation-aware features. Tab. IV reports orientation accuracy across all configurations.

TABLE IV
ORIENTATION PREDICTION ACCURACY (%) ON TEST SET. THE EXTENDED MODEL ACHIEVES BEST ORIENTATION ACCURACY DUE TO INCREASED CAPACITY.

Model	MN10	MN40
Basic	89.2	78.0
Fast	88.4	77.2
Extended	–	78.24

Orientation accuracy is consistently lower than classification accuracy, as predicting the exact 30° rotation bin among 12 options is inherently more challenging than category-level classification. On ModelNet40, the extended 4-layer architecture achieves 78.24% orientation accuracy, slightly outperforming the 2-layer Basic model (78.0%). This suggests the additional convolutional layers improve orientation-specific feature extraction. The Fast variant shows only marginally reduced orientation accuracy (0.8% drop), indicating FC6 compression does not significantly impact the orientation prediction pathway.

E. Per-Class Analysis

To understand where the Fast model loses accuracy, I analyzed per-class performance on ModelNet10. Tab. V shows classes with the largest accuracy gaps.

TABLE V
PER-CLASS ACCURACY (%) ON MN10 TEST SET FOR CLASSES WITH LARGEST BASIC-FAST GAPS.

Class	Basic	Fast	Gap
night stand	86.0	82.0	-4.0
dresser	90.7	87.2	-3.5
desk	91.9	89.5	-2.4
bathtub	92.0	94.0	+2.0

The largest accuracy drops occur for furniture classes with similar geometric structures (night stand, dresser, desk), suggesting reduced FC6 capacity limits fine-grained discrimination. The Fast model slightly outperforms Basic on bathtub, possibly due to reduced overfitting.

F. Training Efficiency

TABLE VI
TRAINING TIME COMPARISON (50 EPOCHS, NVIDIA A:40 GPU).

Model	Dataset	Time (s)	Speedup
Basic	MN10	406.0	–
Fast	MN10	351.0	1.16×
Basic	MN40	972.6	–
Fast	MN40	825.4	1.18×

Tab. VI reports training times for 50 epochs. The Fast variant provides 13–18% speedup due to reduced FC layer computation and smaller gradient tensors.

The speedup is limited because 3D convolutional layers, unchanged between Basic and Fast, dominate computation. On ModelNet40, the Fast variant saves approximately 2.5 minutes per training run.

G. Comparison with Published Results

TABLE VII
COMPARISON WITH PUBLISHED RESULTS ON MODELNET10 AND MODELNET40. MY IMPLEMENTATION USES 50 TRAINING EPOCHS WITH TEST-TIME VOTING.

Method	Layers	Params	MN10	MN40
3D ShapeNets [3]	3	–	83.5	77.0
VoxNet [4]	2	890K	92.0	83.0
ORION [2]	2	910K*	93.8	88.1
ORION [2]	4	4M*	93.9	89.7
Mine (Basic)	2	931K	93.72	87.56
Mine (Fast)	2	482K	93.50	86.63
Mine (Extended)	4	3.29M	–	88.33

*Paper reports 910K/4M; actual codebase yields 931K/3.29M parameters.

Tab. VII compares my results with the original ORION paper and related methods. The baseline closely matches the original 2-layer result (93.72% vs 93.8%), validating the reimplementation.

On ModelNet40, my Basic model achieves 87.56% compared to the original 88.1% (2-layer). This gap is likely attributable to my use of only 50 training epochs, whereas the original work may have trained longer. The extended 4-layer model achieves 88.33%, closer to the reported 89.7%. ORION-Fast achieves accuracy comparable to VoxNet while benefiting from multi-task training.

H. Complexity vs. Performance Trade-off

My experiments provide convergent evidence that the original ORION design is already near-optimal for ModelNet, and that its reported accuracies represent an approximate upper bound for voxel-based architectures on these benchmarks at 32^3 resolution. Three independent lines of evidence support this conclusion. First, all architectural interventions aimed at improving accuracy: SE attention blocks, deeper convolutional stacks, alternative dropout schedules, either failed to improve or actively degraded performance. Second, the extended 4-layer architecture (3.29M parameters) achieves only 0.77% over the 2-layer Basic on MN40, a $3.5\times$ parameter increase for less than 1% gain. Third, PointNet, a fundamentally different architecture with $7.2\times$ more parameters, achieves 86.85% on MN40, below ORION Basic (87.56%), suggesting the accuracy ceiling is a property of the dataset and voxel resolution rather than ORION’s specific design. Conversely, ORION-Fast shows that 48% parameter reduction incurs only 0.93% accuracy loss, confirming the original model was over-parameterized while remaining near this ceiling.

I. Ablation Studies

I conducted comprehensive ablation experiments to understand the contribution of key design choices. All ablations use ModelNet10 unless otherwise specified.

Label Smoothing Impact.

TABLE VIII
EFFECT OF LABEL SMOOTHING FACTOR ON MN10 FAST TEST ACCURACY.

ϵ	0	0.01	0.05	0.1
Fast Acc (%)	92.40	92.73	92.51	92.07

Tab. VIII shows the effect of label smoothing factor ϵ on the Fast variant. The optimal value is $\epsilon = 0.01$, providing regularization without excessive label uncertainty.

Orientation Task Weight. Varying γ in Eq. (3) from 0.3 to 0.7 showed that $\gamma = 0.5$ provides optimal classification accuracy. At $\gamma = 0.3$, reduced orientation learning signal decreased classification accuracy by 0.6%. At $\gamma = 0.7$, over-emphasizing orientation hurt classification by 0.8%.

FC6 Dimension Sweep. Testing FC6 dimensions of 32, 64, 96, and 128 neurons yielded test accuracies of 90.6%, 92.7%, 93.4%, and 93.7% respectively. The dimension of 64 provides the best efficiency-accuracy trade-off: moving to 96 recovers only 0.7% accuracy while increasing parameters by 50%.

Dropout Rate Analysis. I tested dropout rates of 0.2, 0.3, 0.4, and 0.5 on FC layers. The rate of 0.3 (original) performed best; 0.2 led to slight overfitting, while 0.4 and 0.5 caused underfitting with 0.5% and 1.2% accuracy drops respectively.

Learning Rate Schedule. Comparing step decay (epochs 20, 40) versus cosine annealing showed step decay performs 0.4% better, likely due to sharp transitions enabling escape from suboptimal basins.

V. CROSS-ARCHITECTURE COMPARISON WITH POINTNET

To contextualize ORION’s voxel-based approach against a fundamentally different paradigm, I implement PointNet [6] for direct comparison, isolating the effect of data representation (voxel grid vs. point cloud) while controlling for training data, splits, and evaluation protocol.

A. Architecture and Training

PointNet processes raw point clouds through shared MLPs with learned spatial alignment, achieving permutation invariance via global max pooling. The classification head consists of FC layers (512, 256, C) with batch normalization and dropout (0.3). The loss combines cross-entropy with a feature transform regularization term $\lambda \|I - AA^T\|_F$ ($\lambda = 0.001$). Unlike ORION, PointNet has no orientation prediction head and contains 3.47M parameters, approximately $7.2\times$ more than ORION-Fast.

To ensure a controlled comparison, PointNet reads the same HDF5 voxel data as ORION. Each 32^3 voxel grid is converted on-the-fly to a 1024-point cloud by extracting occupied voxel centers, centering, normalizing to the unit sphere, and sampling to a fixed size. During training, Gaussian jitter ($\sigma = 0.01$) is applied. PointNet uses Adam (lr = 0.001, weight decay 10^{-4}) with StepLR decay (factor 0.5 every 20 epochs) for 50 epochs. The same 12-rotation voting protocol (Eq. 5) is used for evaluation.

B. Results and Analysis

TABLE IX
CROSS-ARCHITECTURE COMPARISON ON MODELNET BENCHMARKS WITH TEST-TIME VOTING..

Method	Input	Params	MN10	MN40
PointNet [6]	mesh PC	3.47M	–	89.2
Mine (Basic)	voxel	931K	93.72	87.56
Mine (Fast)	voxel	482K	93.50	86.63
Mine (PointNet)	voxel PC	3.47M	93.17	86.85

Tab. IX compares PointNet with ORION variants under controlled conditions. My PointNet achieves 93.17% on MN10 and 86.85% on MN40 with voting. On MN40, PointNet slightly outperforms ORION-Fast (86.85% vs 86.63%) but falls below ORION Basic (87.56%) despite using $7.2\times$ more parameters (3.47M vs 489K). This result reinforces the evidence from Section IV-H that ORION’s reported accuracies represent an approximate upper bound for this dataset at 32^3 voxel resolution: even a fundamentally different architecture with substantially more parameters cannot surpass ORION Basic. The gap relative to the original PointNet result (89.2% on MN40) is attributable to using voxel-derived point clouds quantized to 32^3 grid resolution rather than mesh-sampled point clouds. Both architectures struggle with geometrically similar furniture classes, though PointNet’s global max pooling discards some fine-grained spatial information that ORION’s 3D convolutions preserve.

VI. CONCLUDING REMARKS

This work presented a PyTorch reimplement of ORION, introduced ORION-Fast achieving 48% parameter reduction with minimal accuracy degradation, and provided a cross-architecture comparison with PointNet. My experiments demonstrate that multi-task learning with orientation prediction remains effective for 3D classification, that significant efficiency gains are achievable through FC layer compression, and that ORION achieves comparable accuracy to PointNet with substantially fewer parameters.

Key Findings. The FC6 layer can be compressed from 128 to 64 neurons with less than 1% accuracy drop, while FC5 is critical. Multiple independent experiments: failed SE attention, marginal gains from $3.5\times$ more parameters in the extended architecture, and PointNet achieving lower accuracy with $7.2\times$ more parameters, converge on the conclusion that the original ORION design is already near-optimal for ModelNet voxelized data, and that reported accuracies represent an approximate upper bound for this dataset at 32^3 voxel resolution.

Limitations. This work focused on ModelNet with synthetic CAD models; real-world performance may differ. The PointNet comparison uses voxel-derived point clouds, possibly underestimating its full potential.

REFERENCES

- [1] N. Sedaghat, M. Zolfaghari, E. Amiri, and T. Brox, “ORION source code,” <https://github.com/lmb-freiburg/orion>, 2017.

- [2] N. Sedaghat, M. Zolfaghari, E. Amiri, and T. Brox, "Orientation-boosted voxel nets for 3d object recognition," in *British Machine Vision Conference (BMVC)*, 2017.
- [3] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3D ShapeNets: A Deep Representation for Volumetric Shapes," *arXiv preprint arXiv:1406.5670*, 2015.
- [4] D. Maturana and S. Scherer, "VoxNet: A 3D Convolutional Neural Network for real-time object recognition," 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Hamburg, Germany, 2015, pp. 922-928, doi: 10.1109/IROS.2015.7353481. keywords: Three-dimensional displays;Object recognition;Laser radar;Sensors;Neural networks;Feature extraction;Robots,
- [5] H. Su, S. Maji, E. Kalogerakis, and E. Learned-Miller, "Multi-view convolutional neural networks for 3d shape recognition," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 945-953, 2015.
- [6] R. Q. Charles, H. Su, M. Kaichun and L. J. Guibas, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 2017, pp. 77-85, doi: 10.1109/CVPR.2017.16. keywords: Three-dimensional displays;Shape;Computer architecture;Feature extraction;Machine learning;Semantics,
- [7] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1135-1143, 2015.
- [8] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7132-7141, 2018.